

c# Programmieren (c) HTL-Leonding

Chess

Lehrziele

- mehrdimensionale Arrays
- Datei Lesen/Schreiben
- Datenkapsel

Aufgabenstellung

Schach ist ein sehr bekanntes Spiel: siehe <https://de.wikipedia.org/wiki/Schach>

Das gesuchte Programm soll das Laden und Speichern eines Schachspiels (+ Validieren) unterstützen. Die zu platzierenden Figuren sind in einer Csv Datei abgelegt. Nach dem Lesen der Csv prüft das Programm, ob ein gültiges "Spielfeld" erstellt werden kann.

Folgende Spielregeln müssen eingehalten werden:

- Die Anzahl der Figuren muss stimmen.
 - Jede Farbe muss exakt einen König haben.
 - Zwei Könige in einer Farbe sind nicht möglich.
 - Erreicht ein Bauer das Ende des Spielfelds, darf er gegen eine beliebig andere Figur getauscht werden. Daher sind 9 Damen (eine vom der Startaufstellung und 8 durch Bauern getauschte) erlaubt. Sind 9 Damen vorhanden, darf es dann aber keinen Bauern mehr geben.
- Die Figuren dürfen nur auf freie Felder platziert werden.
Es ist nicht möglich, unterschiedliche Figuren auf das gleiche Feld zu setzen.
- Ein weisser Bauer darf nicht in der Reihe 1, ein schwarzer Bauer nicht in der Reihe 8 sein. Bauern können ja nur vorwärts fahren!
- Expertenaufgabe: Beide Könige (weiß und schwarz) dürfen nicht im "Schach" stehen.
Hinweis: eigentlich müsste man hier noch beachten, welche Farbe am Zug ist.

Csv-Format

Das Format der Csv Datei ist vorgegeben.

```
Pos;Type;Color
A1;R;W
B1;KN;W
C1;B;W
D1;Q;W
E1;K;W
F1;B;W
G1;KN;W
H1;R;W
```

Die drei Spalten enthalten folgende Werte:

- *D1* steht für die vierte Spalte (D) und erste Reihe (1) - 1 basiert
- Der Type einer Figur kann sein:
 - K - King
 - Q - Queen
 - R - Rook
 - B - Bishop
 - KN - Knight
 - P - Pawn
- Die Farbe ist W (=white) oder B (=black)

Programmablauf

Schreiben Sie ein Programm mit dem Namen **Chess** .

- Die Csv Datei wird entweder durch einen Kommandozeilenparameter angegeben oder, wenn dieser fehlt, wird der Dateiname "**Chess.csv**" verwendet.
- Alle Figuren werden von der Csv Datei in ein Array der Datenkapsel (**ChessPiece**) gelesen.
- Anschließend wird geprüft, ob mit den gelesenen Schachfiguren ein gültiges Spiel erstellt werden kann.
- Ist die Definition gültig, wird das Spielfeld auf der Konsole ausgegeben.
- Bei einer ungültigen Definition wird eine entsprechende Fehlermeldung ausgegeben.

Programmdesign

Achten Sie bei der Umsetzung auf ein sauberes Design Ihres Programms.

In dem bereitgestellten Programm-Template sind Unittests vorhanden. Die Unittests prüfen auch interne Programmfunktionalitäten (Hilfs-Methoden). Diese sind bei der Umsetzung hilfreich - passen Sie das Design daher so an, dass diese Methoden verwendet werden. Änderungen an den *Unittests* sind selbstverständlich nicht erlaubt.

Programmintern können/sollen folgende Punkte umgesetzt sein:

Datenkapsel ChessPiece

- **Row**
Position - Row - des Schachfigur, 0 basiert
- **Col**
Position - Column - des Schachfigur, 0 basiert
- **Type**
Legt (als Ganzzahl) fest, um welche Figur es sich handelt.
 - 1 => König - King
 - 2 => Dame - Queen
 - 3 => Turm - Rook
 - 4 => Läufer - Bishop
 - 5 => Springer - Knight
 - 6 => Bauer - Pawn
- **IsBlack**
Farbe der Figur.

Methoden (Chess.cs)

- `ChessPiece[] ReadFromCsv(string fileName)`
Die Csv Datei wird gelesen und der Inhalt in einem Array der Datenkapsel `ChessPiece` gespeichert. Eine Validierung wird NICHT durchgeführt.
- `ChessPiece?[,]? CreateField(ChessPiece[] pieces)`
Die Methode erstellt das Spielfeld (Array mit einer fixe Größe von 8x8). Jedes durch ein Figur belegte Feld wird mit der Referenz auf die Datenkapsel `ChessPiece` initialisiert, alle anderen Felder bleiben null. Es werden nur gültige Spielfelder erstellt. Im Fehlerfall gibt die Methode `null` als Ergebnis zurück. Gültige Spielfelder sind jene Felder, die die richtige Anzahl von Figuren aufweisen und die Figuren korrekt platziert haben (siehe **Aufgabenstellung**).
- `void Print(ChessPiece[] field)`
Die Methode druckt das Spielfeld auf der Konsole aus. Hinweis: Verwenden sie die Methode `CreateField` um das Spielfeld zu erstellen. Die Ausgabe richtet sich nach dem unten angegebenen Beispiel, siehe **Bildsschirmausgabe**.

Testen Sie das Programm ausführlich.

Bildschirmausgabe

```
Chess Validator
=====
Game from: Game.csv

  A  B  C  D  E  F  G  H
+---+---+---+---+---+---+---+
8| Rb|KNb| Bb| Qb| Kb| Bb|KNb| Rb|8
+---+---+---+---+---+---+---+
7| Pb| Pb| Pb| Pb| Pb| Pb| Pb| Pb|7
+---+---+---+---+---+---+---+
6|  |  |  |  |  |  |  | 6
+---+---+---+---+---+---+---+
5|  |  |  |  |  |  |  | 5
+---+---+---+---+---+---+---+
4|  |  |  |  |  |  |  | 4
+---+---+---+---+---+---+---+
3|  |  |  |  |  |  |  | 3
+---+---+---+---+---+---+---+
2| Pw| Pw| Pw| Pw| Pw| Pw| Pw| Pw|2
+---+---+---+---+---+---+---+
1| Rw|KNw| Bw| Qw| Kw| Bw|KNw| Rw|1
+---+---+---+---+---+---+---+
  A  B  C  D  E  F  G  H
```